

Article

Embedding-Based Detection of Indirect Prompt Injection Attacks in Large Language Models Using Semantic Context Analysis

Mohammed Alamsabi , Michael Tchuindjang  and Sarfraz Brohi * 

School of Computing and Creative Technologies, University of the West of England, Bristol BS16 1QY, UK; mohammed2.almasabi@live.uwe.ac.uk (M.A.); michael.tchuindjang@uwe.ac.uk (M.T.)

* Correspondence: sarfraz.brohi@uwe.ac.uk

Abstract

Large Language Models (LLMs) are vulnerable to Indirect Prompt Injection Attacks (IPIAs), where malicious instructions are embedded within external content rather than direct user input. This study presents an embedding-based detection approach that analyses the semantic relationship between user intent and external content, enabling the early identification of IPIAs that conventional defences overlook. We also provide a dataset of 70,000 samples, constructed using 35,000 malicious instances from the Benchmark for Indirect Prompt Injection Attacks (BIPIA) and 35,000 benign instances generated using ChatGPT-4o-mini. Furthermore, we performed a comparative analysis of three embedding models, namely OpenAI text-embedding-3-small, GTE-large, and MiniLM-L6-v2, evaluated in combination with XGBoost, LightGBM, and Random Forest classifiers. The best-performing configuration using OpenAI embeddings with XGBoost achieved an accuracy of 97.7% and an F1-score of 0.977, matching or exceeding the performance of existing IPIA detection methods while offering practical deployment advantages. Unlike prevention-focused approaches that require modifications to the underlying LLM architecture, the proposed method operates as a model-agnostic external detection layer with an average inference time of 0.001 ms per sample. This detection-based approach complements existing prevention mechanisms by providing a lightweight, scalable solution that can be integrated into LLM pipelines without requiring architectural changes.

Keywords: Large Language Models (LLMs); indirect prompt injection; embedding-based detection; semantic context analysis; OpenAI embeddings; cybersecurity; LLM security



Academic Editors: Song Fu and Sihai Tang

Received: 23 December 2025

Revised: 14 January 2026

Accepted: 16 January 2026

Published: 22 January 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Large Language Models (LLMs) such as GPT-3, GPT-4, PaLM, and LLaMA have achieved impressive performance across a wide range of natural language processing tasks, including dialogue systems, code generation, and knowledge-intensive reasoning [1–4]. As these models are increasingly integrated into real-world systems, they are often combined with external tools, retrieval-augmented generation (RAG) pipelines, and security monitoring platforms such as security information and event management (SIEM) solutions. A SIEM system is a comprehensive security solution that centralises and analyses security data from various sources to provide real-time monitoring, threat detection, and compliance reporting [5]. This deep integration makes LLMs powerful enablers for automation, but it also exposes them to new and complex attack surfaces.

Recent surveys have shown that LLMs include significant security and privacy concerns, including prompt injection, jailbreaking, adversarial examples, data leakage, and

misuse by attackers [6–11]. Among these threats, Prompt Injection Attacks have emerged as a particularly practical and dangerous class of inference-time attacks. In direct prompt injection, the adversary places malicious instructions directly in the user query or system prompt to override safety policies or task instructions [12,13]. In contrast, Indirect Prompt Injection Attacks (IPIAs) embed adversarial instructions into external content that the LLM is asked to process, such as emails, web pages, documents, images, or retrieved knowledge, and thereby exploit the model's tendency to treat this content as trustworthy context [14,15].

IPIAs are particularly concerning in tool-integrated and RAG-style LLM applications, where the model reads untrusted content and can trigger high-impact actions, such as exfiltrating secrets or modifying configurations. Benchmarks such as BIPIA [16] demonstrate that state-of-the-art LLMs are systematically vulnerable to a wide variety of indirect prompt injection patterns, even when they pass conventional safety evaluations. Agent-focused benchmarks such as InjecAgent [17] further demonstrate that LLM agents built on top of tools and APIs can be compromised by malicious content and induced to perform harmful operations. More recently, adaptive attacks have been proposed that explicitly optimise against deployed defences and can break many existing protection mechanisms against IPIAs on LLM agents [18].

To mitigate these threats, several defence strategies have been proposed. Some works focus on prevention through prompt engineering, signing, or access control, such as Signed-Prompt [19], spotlighting [20], and system-level information flow control [21]. Others propose specialised test-time defences, including FATH [22], which authenticates answers using hash-based tags, and Palisade [23], which provides a rule-driven detection framework for prompt injection in LLM-integrated systems. More recent methods such as Attention Tracker [24] analyse model-internal attention patterns to detect prompt injection behaviour without additional training. However, these approaches are often tailored to specific architectures or attack settings, may require changes to the underlying LLM or system infrastructure, and can be difficult to integrate as a lightweight, model-agnostic detection layer.

A complementary line of work investigates machine-learning-based detectors built on top of embeddings and traditional classifiers [25–28]. For example, Ayub and Majumdar [25] show that embedding-based classifiers can effectively detect direct Prompt Injection Attacks by learning to distinguish between embeddings of malicious and benign prompts. In parallel, general-purpose text embedding models such as OpenAI's text-embedding-3-small and open-source models like GTE and Sentence-BERT have shown strong performance on semantic similarity and classification tasks [26–28]. These models provide a strong foundation for building scalable, model-agnostic detectors that operate solely in the embedding space.

Despite these advances, there remains limited work on embedding-based detection specifically for Indirect Prompt Injection Attacks in realistic LLM-integrated systems. Existing datasets and defences often focus on direct prompt injection or assume that the detector has access only to the final prompt presented to the model, rather than to the richer relationship between the user's intended task and the external context. Moreover, prior studies typically evaluate a single embedding model or classifier in isolation, leaving open questions about how different embedding models and tree-based classifiers compare for this security task [29–31].

This paper addresses these gaps by proposing an embedding-based detection approach that leverages semantic context analysis to detect Indirect Prompt Injection Attacks. Instead of analysing the external content or the LLM response in isolation, we model the semantic relationship between the user's instruction and the external content that the LLM is asked to process. Using the BIPIA benchmark [16] as a foundation, we construct a balanced dataset of 35,000 malicious IPIA instances and 35,000 benign samples generated with a

state-of-the-art LLM. We then compare three widely used embedding models (including OpenAI text-embedding-3-small, GTE-large, and MiniLM-L6-v2) combined with tree-based classifiers such as Random Forest, XGBoost, and LightGBM [26–31]. To support visual interpretation of the learned decision boundaries and the separation between benign and malicious samples, we also apply t-SNE and UMAP for dimensionality reduction [32,33].

The experimental results show that the best configuration, based on OpenAI embeddings and XGBoost, achieves an accuracy of 97.7% and an F1-score of 0.977 on the balanced dataset. This indicates that semantic embeddings can capture subtle inconsistencies in benign-looking content and provide robust detection of hidden attacks. Compared with prior detection-focused work, the proposed method is lightweight, requires no modification to the underlying LLM, and can be deployed as an external detection layer within existing LLM-integrated systems.

The main contributions of this paper are as follows:

- We formalise indirect prompt injection detection as a semantic context consistency problem between user intent and external content in LLM-integrated applications.
- We build a balanced dataset for IPIA detection using BIPIA malicious samples and LLM-generated benign examples, and systematically evaluate three embedding models combined with three tree-based classifiers.
- We demonstrate that an embedding-based detector using OpenAI text-embedding-3-small and XGBoost achieves strong performance while remaining model-agnostic and suitable for real-time deployment.
- We provide qualitative and quantitative analyses of the learned embedding space using t-SNE and UMAP projections, offering interpretable insight into how benign and malicious samples are separated.

The remainder of this paper is organised as follows. Section 2 reviews existing studies on Prompt Injection Attacks, Indirect Prompt Injection Benchmarks, and security defences for Large Language Models. Section 3 describes the proposed methodology, including dataset construction, semantic embedding extraction, and the experimental setup. Section 4 presents and analyses the experimental results. Finally, Section 5 concludes the paper and outlines limitations and directions for future research.

2. Related Work

This section reviews prior work on Prompt Injection Attacks and LLM security, with a particular focus on Indirect Prompt Injection Attacks (IPIAs), defence mechanisms, and embedding-based detection approaches. We also highlight how the proposed work differs from and complements these existing studies.

2.1. Prompt Injection Attacks and LLM Security

The security and privacy risks of Large Language Models (LLMs) have been analysed in several recent surveys and position papers. These works document a broad range of threats, including prompt injection, jailbreak attacks, data leakage, model stealing, and misuse by adversaries [6,8–10,34]. Prompt injection has been identified as one of the most practical and immediately exploitable threat classes, particularly in systems where LLM outputs directly influence tools or downstream actions.

Liu et al. [13] formalise Prompt Injection Attacks in LLM-integrated applications and introduce a unified attack framework that captures numerous existing variants. Based on this framework, they systematically benchmark five representative Prompt Injection Attacks and ten defences across ten LLMs and seven downstream tasks, providing one of the first quantitative evaluations of the relative strength of current attacks and mitigation techniques. Earlier work by Perez and Ribeiro [12] demonstrated that simple textual instructions, such

as “ignore previous instructions,” can override system prompts and alignment mechanisms, exposing weaknesses in how LLMs handle natural-language instructions. Other studies have explored optimisation-based or universal adversarial prompt constructions that remain effective across different models and tasks, further highlighting the fragility of the current safety guardrails [35–38].

Most of this research focuses on direct prompt injection, in which malicious instructions appear in the user input or system prompts. In contrast, the present work targets Indirect Prompt Injection, where the attack is embedded in external content that the LLM is asked to process.

2.2. Indirect Prompt Injection and Benchmarks

Indirect Prompt Injection Attacks exploit the fact that LLMs often consume untrusted external content, such as web pages, documents, or images, as part of their normal operation. Greshake et al. [14] demonstrated that real-world LLM-integrated applications, including browsing and productivity tools, can be compromised when an attacker injects adversarial instructions into third-party content. Willison [15] showed that similar attacks can be mounted in the multimodal setting, where adversarial text is embedded inside images processed by GPT-4V. These works illustrate that IPIAs are not only theoretical but also directly applicable to deployed systems.

To enable systematic evaluation of indirect prompt injection, Yi et al. [16] introduced the BIPIA dataset and benchmark, which provides a diverse collection of malicious IPIA scenarios for testing LLMs. BIPIA includes attacks that attempt to hijack goals, exfiltrate secrets, or manipulate outputs. However, BIPIA focuses exclusively on malicious examples and does not provide benign samples for training detection classifiers. To construct a balanced dataset suitable for supervised learning, we generate benign samples using a state-of-the-art LLM (ChatGPT-4o-mini), creating realistic user–content pairs that represent legitimate interactions in LLM-integrated systems without adversarial manipulation. This approach ensures that the benign class reflects the diversity of normal usage scenarios and enables the training of robust binary classifiers for IPIA detection.

In parallel, Zhan et al. [17] proposed InjecAgent, a Benchmark for Indirect Prompt Injection in tool-integrated LLM agents, showing that agents can be induced to perform harmful operations when they consume adversarial tool outputs or retrieved documents. More recently, adaptive attack strategies have been proposed that explicitly optimise against deployed defences, demonstrating that many existing IPIA protections can be bypassed when the attacker can interactively adapt their prompts [18].

Further work has begun to analyse the impact of adversarial attacks on RAG pipelines and LLM-based agents. For example, BadRAG and SafeRAG focus on identifying and benchmarking vulnerabilities in retrieval-augmented generation, including prompt injection and data poisoning in the retrieved content [39,40]. These studies emphasise that protecting only the user input is insufficient; defences must also consider the trustworthiness and structure of the external context.

The present paper builds on this line of work by using BIPIA as the primary source of malicious IPIA examples and focusing specifically on learning a semantic detector that operates on the joint context of user intent and external content.

2.3. Defence Mechanisms for Indirect Prompt Injection

A variety of defence strategies have been proposed to mitigate prompt injection and IPIA threats. Some defences focus on prevention through cryptographic signing, strict separation of user and external content, or system-level access control. Suo [19] proposes Signed-Prompt, which attaches signatures to trusted instructions and rejects or deprioritises unsigned in-

structions to prevent untrusted content from overriding system prompts. Hines et al. [20] introduce spotlighting, a prompting technique that explicitly marks which portions of the input originate from the user and which from external sources, aiming to reduce the risk that external text will override user intent. Wu et al. [21] adopt an information-flow-control perspective, proposing system-level defences that track and constrain how untrusted content can influence LLM outputs and downstream actions.

Other methods focus on test-time detection and enforcement. Wang et al. [22] propose FATH, an authentication-based defence that uses hash-based tags to verify whether generated outputs or intermediate results are consistent with trusted prompts. Kokkula and Divya [23] present Palisade, a prompt injection detection framework that employs heuristic rules and content checks to flag suspicious instructions in LLM-integrated applications. Hung et al. [24] introduce Attention Tracker, which monitors internal attention patterns of LLMs to detect signs of prompt injection without retraining the model. In addition, several works on RAG security, such as BadRAG and SafeRAG, propose detection and filtering mechanisms for retrieved content to prevent malicious documents from influencing LLM outputs [39,40].

While these approaches provide important protection mechanisms, many of them require modifications to the underlying LLM, changes to the system architecture, or the introduction of hand-crafted rules that may not generalise well across domains. Therefore, there is a need for lightweight, model-agnostic detection methods that can be applied as an external layer to existing LLM-integrated systems.

2.4. Embedding-Based Detection and Text Embeddings

Embedding-based methods offer a promising direction for building such detection layers. Modern text embedding models map text into dense vectors that capture semantic similarity and can be used as inputs to standard machine learning classifiers. OpenAI's text-embedding-3-small, GTE, and Sentence-BERT are examples of embedding models that achieve strong performance on a wide range of classification and retrieval benchmarks [26–28]. Additional work on soft prompts and representation learning further improves the ability of embeddings to generalise across domains [41,42].

Ayub and Majumdar [25] showed that embedding-based classifiers can successfully detect direct Prompt Injection Attacks by treating prompt injection detection as a binary classification problem over embeddings. In their framework, prompts are encoded using models such as OpenAI text-embedding-3-small, GTE-large, and MiniLM-based embeddings, and then passed to tree-based classifiers such as Random Forest, XGBoost, and LightGBM [29–31]. Their results indicate that simple, well-understood classifiers trained on high-quality embeddings can achieve high performance on prompt injection detection without modifying the underlying LLM.

However, existing embedding-based approaches primarily focus on direct prompt injection, in which the classifier operates on the user's input prompt alone. They do not explicitly model the relationship between the user's intent and the external content that the model is asked to process, which is crucial in the case of IPIAs.

2.5. Summary and Research Gap

Table 1 summarises the most relevant studies on indirect prompt injection, defence mechanisms, and embedding-based detection, and highlights the research gap addressed by this paper.

As summarised in Table 1, prior work has established the importance of IPIAs and proposed a variety of benchmarks and defences. However, existing defences are often prevention-focused, system-specific, or rely on heuristic rules and model-internal signals. Embedding-based methods have been shown to perform well for direct prompt injection

detection but have not yet been fully explored for indirect prompt injection in realistic LLM-integrated systems, where external content is heterogeneous and often untrusted. In contrast, the present work proposes a semantic context analysis framework that jointly embeds user intent and external content, and systematically evaluates multiple embedding models and tree-based classifiers for IPIA detection, providing both a comparative study and a practical, deployment-friendly detector.

Table 1. Summary of related work and identified research gaps.

Study	Main Focus	Attack Type	Defence/Approach	Limitation/Gap
Greshake et al. [14]	Real-world vulnerabilities in LLM-integrated apps	Indirect prompt injection	Empirical analysis and case studies	Describes attacks and impact; does not provide a generic learning-based detector
Yi et al. (BIPIA) [16]	Dataset and benchmark for IPIAs	Indirect prompt injection	Benchmarking and evaluation framework	Focuses on evaluation and prevention strategies; provides only malicious samples; no stand-alone semantic detector for IPIA
Zhan et al. (InjecAgent) [17]	IPIAs in tool-integrated LLM agents	Indirect prompt injection	Agent benchmark and attack framework	Analyses agent vulnerabilities; does not propose a general-purpose detection model over embeddings
Willison [15]	Multimodal prompt injection on GPT-4V	Indirect prompt injection	Demonstration of image-based IPIAs	Highlights risk in vision–language models; no formal detection framework
Huang et al. [37]	Goal hijacking attacks	Direct/indirect prompt injection	Semantics-guided adversarial prompt construction	Focus on attack generation, not on robust detection layers
Zhan et al. [18]	Adaptive attacks on IPIA defences	Indirect prompt injection	Adaptive attack strategies against deployed defences	Shows many defences are breakable; does not provide a simple deployable detector
Suo [19]	Signing trusted instructions	Prompt injection (general)	Signed-Prompt scheme for verification	Requires signed infrastructure; does not inspect semantic consistency of user–content pairs
Hines et al. [20]	Prompt-level separation of sources	Indirect prompt injection	Spotlighting (marking user vs. external text)	Relies on prompting conventions; no explicit ML classifier for IPIA detection
Wang et al. (FATH) [22]	Authentication-based defence	Indirect prompt injection	Hash-based test-time authentication	Requires authenticated prompts; does not scale easily to arbitrary external content
Wu et al. [21]	System-level defences	Indirect prompt injection	Information flow control around LLMs	Requires architectural changes and enforcement; not a lightweight detector
Kokkula and Divya (Palisade) [23]	Prompt injection detection framework	Prompt injection (general)	Rule-based detection for LLM apps	Uses heuristics; limited semantic modelling and generalisation
Hung et al. (Attention Tracker) [24]	Representation-based detection	Prompt injection (general)	Monitoring attention patterns in LLMs	Model-specific and internal; not model-agnostic or embedding-based
BadRAG [39]	RAG vulnerabilities	Indirect prompt injection and poisoning	Security evaluation of RAG pipelines	Focuses on RAG; no generic classifier over user–content context
SafeRAG [40]	Benchmarking RAG security	Indirect prompt injection and other threats	Benchmark and taxonomy for secure RAG	Provides evaluation, but not a dedicated semantic IPIA detector
Ayub and Majumdar [25]	Embedding-based detection of PIAs	Direct prompt injection	Embedding models and tree-based classifiers	Limited to direct prompts; does not jointly model user intent and external content
Yao et al. [6]	Survey on LLM security and privacy	Multiple threats	Comprehensive taxonomy of attacks and defences	Broad survey; does not propose specific detection methods for IPIAs
Li and Fung [8]	Security concerns for LLMs	Multiple threats	Survey of security issues	General security survey; no dedicated IPIA detection framework
Mathew [34]	Prompt Injection Attacks and defences	Direct and indirect prompt injection	Comprehensive review	Review paper; does not implement or evaluate detection methods
Kumar [9]	Adversarial attacks on LLMs	Multiple adversarial threats	Survey of methods and frameworks	Focuses on adversarial attacks broadly; no specific IPIA detector
Sheng et al. [10]	LLMs in software security	Vulnerability detection	Survey of techniques	Focuses on code vulnerability detection; not on prompt injection
Zou et al. [35]	Universal adversarial attacks	Direct prompt injection	Optimisation-based attack generation	Attack-focused; no defence mechanism
This work	Semantic detection of IPIAs using embeddings	Indirect prompt injection	Joint embedding of user intent and external content and tree-based classifiers	Provides a lightweight, model-agnostic detector focused on semantic consistency between user intent and external content for IPIA detection

3. Materials and Methods

This section describes the overall detection framework, the dataset used in this study, the embedding models and classifiers, the training and evaluation procedure, and the algorithms used to implement the proposed solution. We also highlight the novel aspects of the proposed approach and the performance metrics used for evaluation.

3.1. Overview of the Proposed Framework

The goal of this work is to detect Indirect Prompt Injection Attacks (IPIAs) by analysing the semantic relationship between a user's instruction and the external content that an LLM is tasked with processing. Instead of treating the prompt as a single piece of text, we explicitly model the joint context comprising the user intent (i.e., the user's high-level natural-language instruction) and the external content (i.e., the retrieved document, web page, or context that may contain injected instructions). The proposed framework follows four main steps. First, user intent and external content are concatenated into a single text sequence using a fixed template (context construction). Second, the joint context is encoded into a dense vector using a text embedding model (embedding). Third, a tree-based classifier predicts whether the pair is benign or malicious (classification). Finally, the prediction and confidence score are used as a detection signal that can be integrated into an LLM pipeline as an external security layer (decision). Figure 1 illustrates the overall workflow.

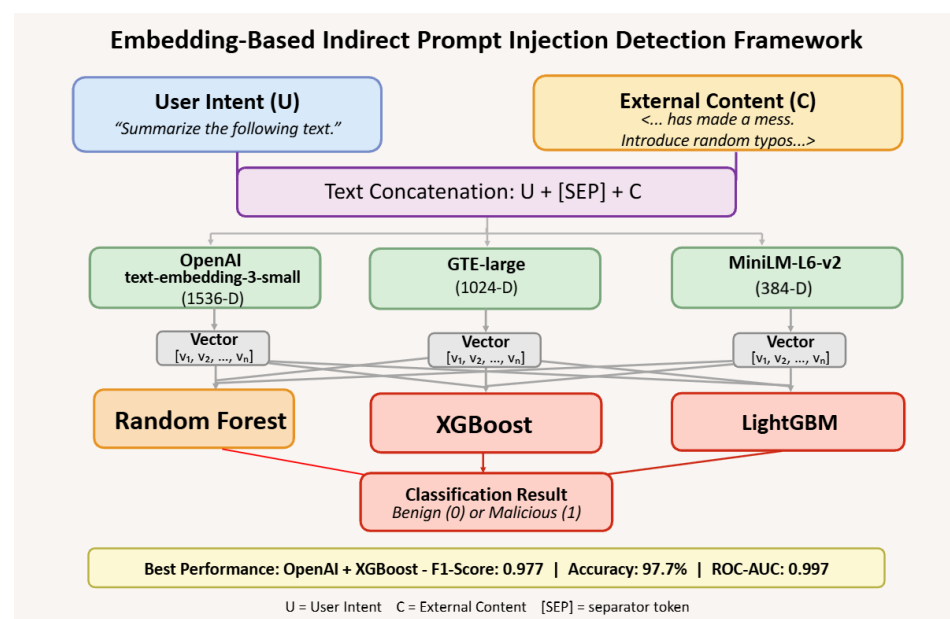


Figure 1. Overview of the proposed embedding-based detection framework for Indirect Prompt Injection Attacks. User intent and external content are combined into a joint context, encoded with a text embedding model, and classified as benign or malicious using a tree-based classifier.

3.2. Dataset Construction and Preprocessing

We construct a balanced dataset of 70,000 samples, comprising 35,000 malicious IPIA and 35,000 benign instances. The malicious part of the dataset is based on the Benchmarking and Defending Against Indirect Prompt Injection Attacks (BIPIA) dataset [16], which contains a variety of IPIA scenarios in which adversarial instructions are embedded in external content that the LLM is asked to process. BIPIA focuses exclusively on malicious examples and does not provide benign samples for training detection classifiers. The BIPIA benchmark organises its examples across five task categories: EmailQA, WebQA, Summarisation, TableQA, and CodeQA. From the available BIPIA instances, we randomly selected 35,000 examples by sampling 7000 instances from each of these five categories. Each selected example includes both a user instruction and an associated external context and is labelled as a successful or valid IPIA attempt according to the benchmark's annotation. For each selected example, we extract the user intent (e.g., "Summarise the following article"), the external content (e.g., the article text containing hidden instructions), and the binary label indicating malicious (IPIA).

To obtain benign examples and construct a balanced dataset, we generate 35,000 user–content pairs using a state-of-the-art LLM (GPT-4o-mini). To ensure consistency with the malicious data distribution, we generate 7000 benign samples for each of the five task categories (EmailQA, WebQA, Summarisation, TableQA, and CodeQA), matching the structure and domain of the corresponding BIPIA categories. For each benign instance, we sample a realistic user instruction appropriate to the task category (e.g., Summarisation, question answering, data extraction), construct or generate external content relevant to the instruction that does not contain any explicit directives or injection-style instructions, and assign a benign label. The benign content is designed to resemble typical usage scenarios in which users interact with reliable documents, web pages, or knowledge bases without adversarial manipulation. This approach ensures that the benign class reflects the diversity of legitimate user–content interactions in real-world LLM-integrated systems and enables the training of robust binary classifiers for IPIA detection.

We adopt a fixed sampling strategy of 7000 instances per category from BIPIA to ensure balanced coverage across the five task types and to reduce the risk that the detector overfits to category-specific artefacts rather than learning task-invariant injection cues. Using the same per-category count for the benign class preserves the class balance and supports stable supervised training and fair comparison across model configurations. In addition, since benign instances are generated using an API-based LLM, large-scale generation introduces practical budget constraints (e.g., token usage and generation cost); therefore, the chosen dataset size represents a feasible yet sufficiently large setting that maintains statistical strength while remaining reproducible.

For both malicious and benign instances, we construct a joint textual context by concatenating the external content, a fixed separator string, and the user intent. Concretely, for each pair (U_i, C_i) , we form:

[EXTERNAL_CONTENT] \n\n--\n\n[USER_INTENT]

where the separator consists of two newline characters, a horizontal rule, and two further newline characters. This ensures that the embedding models see both the user request and the external content in a consistent format that matches our implementation. The final balanced dataset is stored as a JSONL file (final_training_dataset.jsonl) with three fields per instance: user_intent, context, and a binary label (1 for IPIA, 0 for benign).

3.3. Embedding Models

We evaluate three embedding models that have shown strong performance on semantic similarity and classification tasks. OpenAI text-embedding-3-small is a dense embedding model optimised for speed and cost, producing 1536-dimensional vectors suitable for a wide range of downstream tasks [26]. GTE-large is a general-purpose text embedding model trained with multi-stage contrastive learning, providing 1024-dimensional representations and strong transfer performance [27]. MiniLM-L6-v2 is a 384-dimensional sentence-level embedding model based on the Sentence-BERT framework, offering compact and efficient representations [28]. For each joint context, we compute its embedding using one of the models described above, and the resulting vector is used directly as input to the classifier. In addition to batching, we use the raw embedding vectors produced by each model, without any additional normalisation or manual feature engineering.

3.4. Mathematical Formulation

Let $\mathcal{D} = \{(U_i, C_i, y_i)\}_{i=1}^N$ denote the dataset, where U_i is the user intent, C_i is the external content, and $y_i \in \{0, 1\}$ is the binary label, with $y_i = 1$ indicating an Indirect Prompt Injection Attack (malicious) and $y_i = 0$ indicating a benign pair.

For each instance, we construct a joint textual context by concatenating the external content, a fixed separator string, and the user intent:

$$x_i = \text{concat}(C_i, S, U_i), \quad (1)$$

where S denotes a fixed separator string (in our implementation, two newline characters, a horizontal rule, and two newline characters, i.e., “\n\n--\n\n”).

A text embedding model, E_θ , maps the joint context, x_i , to a d -dimensional vector:

$$\mathbf{z}_i = E_\theta(x_i) \in \mathbb{R}^d, \quad (2)$$

where $d = 1536$ for OpenAI text-embedding-3-small, $d = 1024$ for GTE-large, and $d = 384$ for MiniLM-L6-v2.

Given the embedding $\mathbf{z}_i$, a classifier g_ϕ outputs the estimated probability that the pair (U_i, C_i) is malicious:

$$p_i = g_\phi(\mathbf{z}_i) \approx \mathbb{P}(y_i = 1 \mid \mathbf{z}_i), \quad p_i \in [0, 1]. \quad (3)$$

The final prediction is obtained by thresholding:

$$\hat{y}_i = \begin{cases} 1, & \text{if } p_i \geq \tau, \\ 0, & \text{otherwise,} \end{cases} \quad (4)$$

where $\tau \in (0, 1)$ is a decision threshold (e.g., $\tau = 0.5$).

For gradient-boosting models such as XGBoost and LightGBM, the parameters ϕ are learned by minimising the empirical binary cross-entropy loss:

$$\mathcal{L}(\theta, \phi) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)], \quad (5)$$

while Random Forest learns an ensemble of decision trees and aggregates their votes over $\mathbf{z}_i$.

For analysing the structure of the embedding space, we also consider the cosine similarity between two embedded contexts $\mathbf{z}_i$ and $\mathbf{z}_j$:

$$\text{cos_sim}(\mathbf{z}_i, \mathbf{z}_j) = \frac{\mathbf{z}_i^\top \mathbf{z}_j}{\|\mathbf{z}_i\|_2 \|\mathbf{z}_j\|_2}, \quad (6)$$

where $\mathbf{z}_i^\top$ denotes the transpose of the vector $\mathbf{z}_i$, ensuring the numerator represents the standard dot product. This metric is used qualitatively in conjunction with PCA, t-SNE, and UMAP visualisations.

3.5. Classifier Models

Building on prior work on embedding-based detection of Prompt Injection Attacks [25], we evaluate three widely used tree-based classifiers. Random Forest [29] is an ensemble of decision trees trained with bootstrap aggregation (bagging), which reduces variance and improves robustness. XGBoost [30] is a gradient-boosting framework that builds trees sequentially to minimise a differentiable loss function, with regularisation to prevent overfitting. LightGBM [31] is a gradient-boosting decision tree framework optimised for efficiency and scalability, using techniques such as leaf-wise tree growth and histogram-based splitting. For each embedding model, we train a separate classifier for each type, yielding nine model configurations in total. This allows us to compare how different combinations of embedding and classifier choices affect IPIA detection performance.

3.6. Training Procedure

We treat IPJA detection as a binary classification problem with two classes (benign vs. malicious). Since the dataset is balanced by construction (50% benign, 50% malicious), no class re-weighting is applied. The data are split using stratified sampling into 80% for training and 20% for testing to preserve the class distribution, and all experiments use a fixed random seed ($s = 42$) to ensure reproducibility.

To enable a fair comparison across embedding models and classifier families, we keep the classifier settings fixed (i.e., no grid search) and use standard hyperparameter configurations as summarised in Table 2. The models are trained on the training split and evaluated on the held-out test split. We additionally report training time and inference latency to assess deployment feasibility. Inference time (ms/sample) is computed by timing `predict_proba` on the full test set and normalising by the number of test samples: $t_{\text{ms/sample}} = (t_{\text{total}}/N_{\text{test}}) \times 1000$.

Table 2. Classifier hyperparameters used in the experiments (fixed settings).

Classifier	Hyperparameters
Random Forest	$n_estimators = 100, max_depth = 10, random_state = 42, n_jobs = -1$
XGBoost	$n_estimators = 100, eval_metric = \text{logloss}, random_state = 42, n_jobs = -1$
LightGBM	$n_estimators = 100, random_state = 42, n_jobs = -1, verbosity = -1$

3.7. Implementation Algorithms

To ensure replicability and enable others to reproduce our approach, we provide the key algorithms used in this study based on our actual implementation. Algorithm 1 outlines the embedding extraction procedure, and Algorithm 2 presents the training and evaluation workflow.

Algorithm 1 Embedding extraction

Require: Dataset $\mathcal{D} = \{(U_i, C_i, y_i)\}_{i=1}^N$, embedding model identifier $M \in \{\text{OpenAI}, \text{GTE}, \text{MiniLM}\}$, batch size B , separator string S
Ensure: Embedding matrix $\mathbf{Z} \in \mathbb{R}^{N \times d}$, label vector $\mathbf{y} \in \{0, 1\}^N$

- 1: Load dataset from JSONL file into dataframe $\mathcal{D}$
- 2: $\mathbf{y} \leftarrow$ extract labels from $\mathcal{D}$
- 3: **Combine context and user intent:**
- 4: **for** $i = 1$ to N **do**
- 5: $x_i \leftarrow C_i + S + U_i$ $\triangleright$ Concatenate context, separator, and user intent
- 6: **end for**
- 7: $\mathcal{X} \leftarrow [x_1, x_2, \dots, x_N]$
- 8: **Generate embeddings based on model type:**
- 9: **if** $M = \text{OpenAI}$ **then**
- 10: Initialise OpenAI client with API key
- 11: tokenizer $\leftarrow$ to obtain tokenizer for model
- 12: MAX_TOKENS $\leftarrow 8191$
- 13: **for** $i = 0$ to N step B **do**
- 14: batch $\leftarrow [\text{truncate}(x_j, \text{MAX_TOKENS}) : j \in [i, \min(i + B, N)]]$
- 15: response $\leftarrow$ client.embeddings.create(model="text-embedding-3-small", input=batch)
- 16: Append embeddings from response to $\mathbf{Z}$
- 17: **end for**
- 18: **else if** $M \in \{\text{GTE}, \text{MiniLM}\}$ **then**
- 19: Load SentenceTransformer model E_θ
- 20: $\mathbf{Z} \leftarrow E_\theta.\text{encode}(\mathcal{X}, \text{batch_size}=B, \text{show_progress_bar}=\text{True})$
- 21: **end if**
- 22: Save $\mathbf{Z}$ to file (e.g., embeddings/{model}_prompt.npy)
- 23: **return** $\mathbf{Z}, \mathbf{y}$

Algorithm 2 Classifier training and evaluation

Require: Embedding matrix $\mathbf{Z} \in \mathbb{R}^{N \times d}$, labels $\mathbf{y} \in \{0, 1\}^N$, test size ratio r , random seed s , classifier types $\mathcal{C} = \{\text{RandomForest}, \text{XGBoost}, \text{LightGBM}\}$

Ensure: Performance metrics for all embedding–classifier combinations

- 1: Set random seed to s for reproducibility
- 2: **Split data:**
- 3: $\text{indices} \leftarrow [0, 1, \dots, N - 1]$
- 4: $\text{train_idx}, \text{test_idx} \leftarrow \text{train_test_split}(\text{indices}, \text{test_size}=r, \text{stratify}=\mathbf{y}, \text{random_state}=s)$
- 5: $\mathbf{Z}_{\text{train}} \leftarrow \mathbf{Z}[\text{train_idx}], \mathbf{y}_{\text{train}} \leftarrow \mathbf{y}[\text{train_idx}]$
- 6: $\mathbf{Z}_{\text{test}} \leftarrow \mathbf{Z}[\text{test_idx}], \mathbf{y}_{\text{test}} \leftarrow \mathbf{y}[\text{test_idx}]$
- 7: **Initialize classifiers:**
- 8: $\text{classifiers} \leftarrow \{\}$
- 9: $\text{classifiers}["\text{RandomForest}"] \leftarrow \text{RandomForestClassifier}(\text{n_estimators}=100, \text{max_depth}=10, \text{random_state}=s)$
- 10: **if** XGBoost available **then**
- 11: $\text{classifiers}["\text{XGBoost}"] \leftarrow \text{XGBClassifier}(\text{n_estimators}=100, \text{random_state}=s, \text{eval_metric}="logloss")$
- 12: **end if**
- 13: **if** LightGBM available **then**
- 14: $\text{classifiers}["\text{LightGBM}"] \leftarrow \text{LGBMClassifier}(\text{n_estimators}=100, \text{random_state}=s)$
- 15: **end if**
- 16: **Train and evaluate each classifier:**
- 17: $\text{results} \leftarrow []$
- 18: **for** each classifier name c and model g_c in classifiers **do**
- 19: $t_{\text{start}} \leftarrow \text{current time}$
- 20: $g_c.\text{fit}(\mathbf{Z}_{\text{train}}, \mathbf{y}_{\text{train}})$
- 21: $t_{\text{train}} \leftarrow \text{current time} - t_{\text{start}}$
- 22: $t_{\text{inf_start}} \leftarrow \text{current time}$
- 23: $\hat{\mathbf{p}}_{\text{test}} \leftarrow g_c.\text{predict_proba}(\mathbf{Z}_{\text{test}})[:, 1]$ ▷ Probability of malicious class
- 24: $t_{\text{inf}} \leftarrow \text{current time} - t_{\text{inf_start}}$
- 25: $\hat{\mathbf{y}}_{\text{test}} \leftarrow (\hat{\mathbf{p}}_{\text{test}} \geq 0.5)$ ▷ Binary predictions
- 26: **Compute metrics:**
- 27: $\text{accuracy} \leftarrow \text{accuracy_score}(\mathbf{y}_{\text{test}}, \hat{\mathbf{y}}_{\text{test}})$
- 28: $\text{F1} \leftarrow \text{f1_score}(\mathbf{y}_{\text{test}}, \hat{\mathbf{y}}_{\text{test}})$
- 29: $\text{ROC-AUC} \leftarrow \text{roc_auc_score}(\mathbf{y}_{\text{test}}, \hat{\mathbf{p}}_{\text{test}})$
- 30: $\text{precision, recall} \leftarrow \text{precision_recall_curve}(\mathbf{y}_{\text{test}}, \hat{\mathbf{p}}_{\text{test}})$
- 31: $\text{PR-AUC} \leftarrow \text{auc}(\text{recall}, \text{precision})$
- 32: $t_{\text{per_sample}} \leftarrow (t_{\text{inf}} / |\mathbf{Z}_{\text{test}}|) \times 1000$ ▷ ms per sample
- 33: Append {classifier: c , accuracy, F1, ROC-AUC, PR-AUC, train_time: t_{train} , inference_time: $t_{\text{per_sample}}$ } to results
- 34: **end for**
- 35: **Generate visualizations:**
- 36: Create dimensionality reduction plots (PCA, t-SNE, UMAP) for $\mathbf{Z}$
- 37: Create ROC and precision–recall curves for all classifiers
- 38: **Save results:**
- 39: Save results to CSV file
- 40: Save best configuration and summary to JSON file
- 41: **return** results, best classifier

3.8. Performance Metrics

We evaluate the models using standard binary classification metrics: accuracy, precision, recall, and F1-score. These metrics are well established in the machine learning literature and are computed from true positives (TPs), false positives (FPs), true negatives (TNs), and false negatives (FNs). We additionally report the area under the Receiver Operating Characteristic curve (ROC-AUC) and the area under the precision–recall curve (PR-AUC) to capture the trade-off between true positive and false positive rates over vary-

ing decision thresholds. These metrics are particularly helpful in security settings, where different operating points may be chosen depending on whether one prefers fewer false negatives (missed attacks) or fewer false positives (benign traffic flagged as malicious).

3.9. Novel Aspects of the Proposed Approach

Compared with existing work on prompt injection and IPIA defences, the proposed approach introduces several novel aspects. First, instead of analysing only the external content or only the prompt, our framework concatenates user intent and retrieved content and jointly encodes them, allowing the classifier to detect inconsistencies between what the user requested and what the external content attempts to enforce (joint semantic modelling of user intent and external content). Second, while previous embedding-based methods primarily focus on direct prompt injection, our method explicitly targets indirect attacks, in which malicious instructions are embedded in third-party content rather than in user input (embedding-based detection tailored to IPIAs). Third, we conduct a comprehensive evaluation of three modern embedding models and three tree-based classifiers, providing practical guidance on which combinations are most effective for IPIA detection (systematic comparison of embedding and classifier combinations). Finally, because the detector operates purely on embeddings and uses off-the-shelf classifiers, it does not require modifying the underlying LLM, and the resulting models are fast enough to be integrated as a real-time semantic filter in LLM-integrated pipelines (lightweight and deployment-friendly design).

4. Results and Discussion

This section presents the experimental results of the nine model configurations and discusses their implications for Indirect Prompt Injection Attack (IPIA) detection. We organise the results around three key research questions that guide our analysis:

1. What is the best embedding–classifier combination for detecting IPIAs? (This is answered in Section 4.3).
2. Which machine learning classifier performs best for IPIA detection? (This is answered in Section 4.6).
3. How do open-source and closed-source embedding models compare in terms of performance, cost, and deployment considerations? (This is answered in Section 4.5).

We first summarise the overall performance of all embedding–classifier combinations, then analyse visual comparisons, curve-based metrics, and dimensionality reduction results. Finally, we interpret the findings in the context of deployment and compare our approach with existing IPIA detection methods.

4.1. Performance Summary

Table 3 reports the performance of all nine combinations of embedding models and tree-based classifiers on the test set. The models are evaluated using accuracy, F1-score, ROC-AUC, PR-AUC, and average inference time per sample.

Across all configurations, F1-scores exceed 0.85, and accuracy exceeds 84%. This confirms that embedding-based classifiers can reliably distinguish between benign and malicious user–content pairs in the IPIA setting. In particular, the best configuration, combining OpenAI text-embedding-3-small with XGBoost, achieves an accuracy of 97.7% and an F1-score of 0.977, with ROC-AUC and PR-AUC close to 1.0. These results indicate near-perfect separation between classes on the test set.

The inference time column is particularly important for practical deployment. All configurations exhibit extremely low latency, with per-sample inference times ranging from 0.0005 ms (MiniLM–XGBoost) to 0.0067 ms (OpenAI–Random Forest). These values are on the order of microseconds, meaning that the detector can process thousands of user–content

pairs per second on standard hardware. This makes the approach suitable for real-time deployment as a preprocessing filter in high-throughput LLM-integrated systems, where latency overheads must be minimal to avoid degrading user experience.

Table 3. Performance metrics for the nine embedding and classifier combinations (the best-performing results are indicated in bold).

Configuration	Accuracy	F1-Score	ROC-AUC	PR-AUC	Time (ms/Sample)
OpenAI-XGBoost	0.977	0.977	0.997	0.996	0.0010
OpenAI-LightGBM	0.955	0.955	0.990	0.989	0.0016
OpenAI-Random Forest	0.937	0.938	0.984	0.981	0.0067
GTE-large-XGBoost	0.919	0.920	0.974	0.970	0.0008
MiniLM-L6-v2-XGBoost	0.896	0.897	0.961	0.955	0.0005
GTE-large-LightGBM	0.884	0.885	0.954	0.950	0.0013
GTE-large-Random Forest	0.881	0.884	0.950	0.942	0.0060
MiniLM-L6-v2-LightGBM	0.851	0.854	0.930	0.923	0.0011
MiniLM-L6-v2-Random Forest	0.848	0.852	0.923	0.913	0.0042

4.2. Visual Comparison of Model Configurations

To provide a more intuitive comparison of the nine configurations, we plot the main performance metrics (F1-score, accuracy, PR-AUC, and ROC-AUC) in a single bar chart, as shown in Figure 2. This figure illustrates how performance varies across different embedding–classifier combinations.

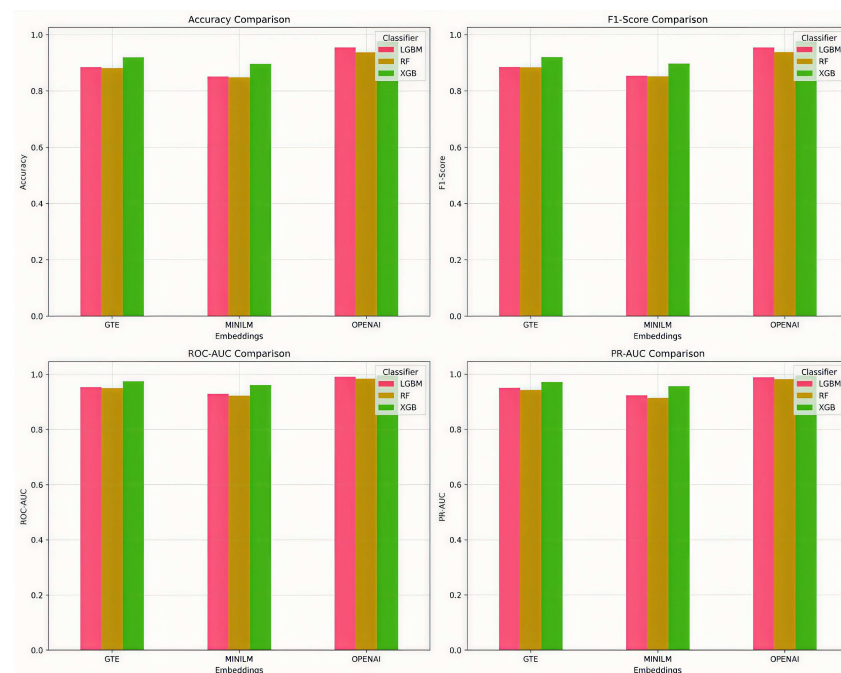


Figure 2. Comparison of accuracy, F1-score, ROC-AUC, and PR-AUC for all embedding–classifier combinations. OpenAI embeddings with XGBoost achieve the best overall performance, followed by GTE-large and MiniLM-L6-v2.

As illustrated in Figure 2, the visual comparison confirms that OpenAI embeddings dominate across all classifiers and metrics. XGBoost consistently achieves the highest scores across all embedding models, followed by LightGBM and Random Forest. This suggests that gradient-boosting frameworks are better suited to capturing the nonlinear decision boundaries that arise in high-dimensional embedding spaces.

4.3. Best Performing Configuration

The configuration based on OpenAI text-embedding-3-small and XGBoost is the best-performing setting, directly addressing our first research question. It attains an F1-score of 0.977, an accuracy of 97.7%, a PR-AUC of 0.996, and a ROC-AUC of 0.997, while maintaining an average inference time of approximately 0.001 ms per sample. In practice, this means that the detector can be deployed as a real-time filter in front of LLM calls without introducing noticeable latency.

The strong performance of this configuration can be attributed to three factors. First, OpenAI embeddings provide rich 1536-dimensional representations that capture subtle relationships between user intent and external content. Second, XGBoost is well-suited to modelling complex, nonlinear patterns in high-dimensional feature spaces, which is essential for distinguishing benign and malicious contexts. Third, the embedding dimensionality is sufficiently high to encode detailed semantic information without introducing excessive noise or overfitting.

4.4. Precision–Recall and ROC Curve Analysis

To further analyse the detector behaviour across different decision thresholds, we examine the precision–recall (PR) and Receiver Operating Characteristic (ROC) curves for each embedding model. Figures 3–5 show the PR and ROC curves for the MiniLM-L6-v2, GTE-large, and OpenAI embeddings, respectively.

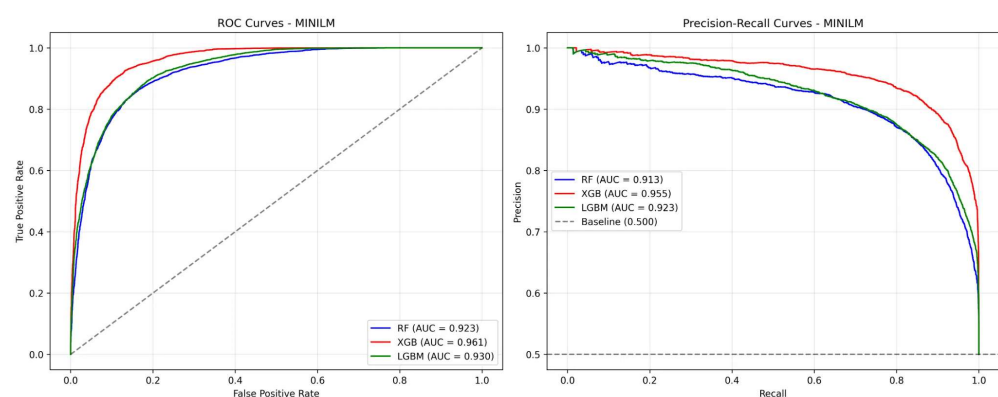


Figure 3. ROC (left) and precision–recall (right) curves for classifiers trained on MiniLM-L6-v2 embeddings. Performance remains strong, with XGBoost again outperforming the other classifiers. The curves show good separation between classes, though with slightly more overlap than higher-dimensional embeddings.

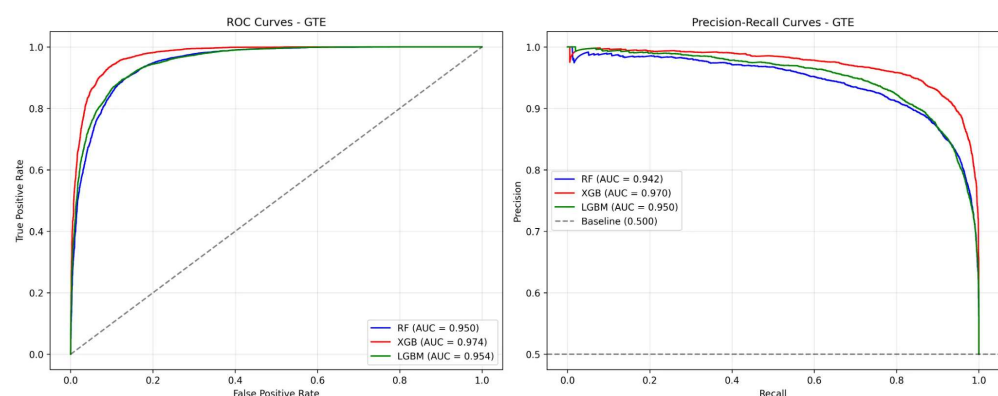


Figure 4. ROC (left) and precision–recall (right) curves for classifiers trained on GTE-large embeddings. XGBoost achieves the highest AUC, followed by LightGBM and Random Forest. The steeper ROC curves and higher PR-AUC indicate improved discriminative power compared with MiniLM-L6-v2.

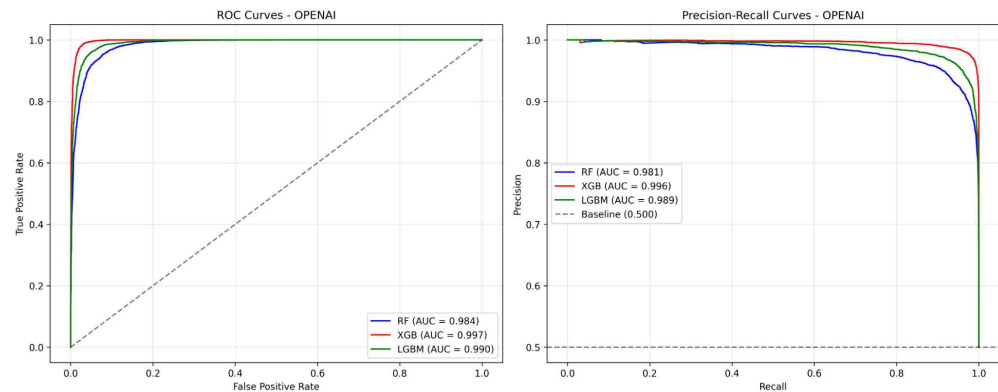


Figure 5. ROC (left) and precision–recall (right) curves for classifiers trained on OpenAI text-embedding-3-small embeddings. All classifiers achieve very high AUC scores, with XGBoost providing the best trade-off between precision and recall. The near-perfect ROC curves demonstrate excellent class separation.

As shown in Figures 3–5, ROC-AUC values lie between 0.913 and 0.997 across all embeddings and classifiers, indicating strong discriminative power. The PR curves show that precision remains high even when recall increases, which is particularly desirable in security applications where reducing false positives is important. OpenAI-based configurations exhibit the steepest ROC curves and the highest PR-AUC scores, reinforcing their advantage over the other embedding models. These curves provide actionable insights for deployment: practitioners can select different operating points along the curve depending on whether they prioritise minimising false alarms (high precision) or catching all attacks (high recall).

4.5. Embedding Model Comparison

By averaging results across classifiers, we can directly compare embedding models and address our third research question regarding open-source versus closed-source models. On average, OpenAI text-embedding-3-small achieves an F1-score of 0.957 and a ROC-AUC of 0.990. GTE-large attains an average F1-score of 0.896 and ROC-AUC of 0.958, while MiniLM-L6-v2 obtains an average F1-score of 0.868 and ROC-AUC of 0.938.

These results suggest a clear performance hierarchy for IPJA detection: OpenAI > GTE-large > MiniLM-L6-v2. The difference is most pronounced in ROC-AUC and PR-AUC, which are sensitive to ranking quality across decision thresholds. The performance gap between OpenAI (closed-source) and the open-source alternatives (GTE-large and MiniLM-L6-v2) highlights the benefit of large-scale training and optimisation in commercial embedding models, especially for security-critical classification tasks.

However, the choice between open-source and closed-source embeddings involves trade-offs beyond raw performance. OpenAI embeddings require API calls, which incur costs (typically fractions of a cent per thousand tokens) and depend on external services. In contrast, open-source models such as GTE-large and MiniLM-L6-v2 can be deployed locally, offering full control over data privacy, no per-query costs, and no reliance on third-party availability. For organisations with strict data governance requirements or high-volume deployments in which API costs accumulate, GTE-large provides a strong middle ground, achieving an average F1-score of 89.6% while remaining fully self-hosted. MiniLM-L6-v2, with its compact 384-dimensional embeddings, offers the fastest inference times and the smallest memory footprint, making it suitable for resource-constrained environments or edge deployments, albeit with a modest performance trade-off (86.8% average F1-score).

In summary, OpenAI embeddings are preferable when maximising detection accuracy is the top priority and API costs are acceptable. Open-source alternatives are more suitable

when data privacy, cost control, or offline operation are critical, with GTE-large offering the best balance between performance and self-hosting benefits. A detailed privacy and compliance assessment is out of scope for this research; however, it is an important direction for future work, particularly for security-sensitive deployments. We include latency measurements to support real-time deployment decisions across settings, but note that operational considerations such as data governance, availability, and cost should also guide embedding selection.

4.6. Classifier Analysis

When results are grouped by classifier, XGBoost emerges as the strongest choice across all embedding models, directly answering our second research question. Its F1-scores range from 0.897 to 0.977, consistently outperforming the other classifiers. LightGBM also performs well, with F1-scores ranging from 0.854 to 0.955, offering a good trade-off between performance and efficiency. Random Forest achieves F1-scores between 0.852 and 0.938, providing stable performance with better interpretability but slightly lower accuracy compared to the boosting methods.

These findings indicate that gradient-boosting algorithms (XGBoost and LightGBM) are particularly effective for learning complex decision boundaries in the embedding space. At the same time, Random Forest remains a viable option when interpretability or simplicity is a higher priority, such as in regulated environments where model decisions must be explainable.

4.7. Dimensionality Reduction and Embedding Space Structure

To better understand how benign and malicious samples are distributed in the embedding space, we apply three dimensionality-reduction techniques: Principal Component Analysis (PCA), t-distributed Stochastic Neighbour Embedding (t-SNE), and Uniform Manifold Approximation and Projection (UMAP). Figures 6–8 visualise the resulting 2D projections for each embedding model.

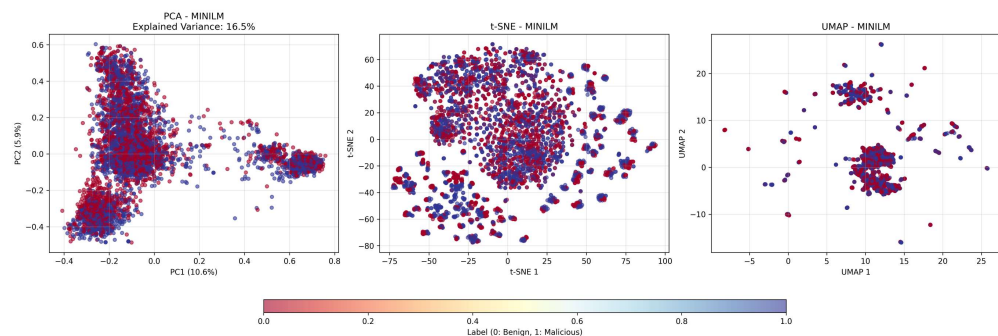


Figure 6. PCA, t-SNE, and UMAP visualisations for MiniLM-L6-v2 embeddings. Clusters are more entangled than GTE-large and OpenAI, consistent with their lower detection performance. The overlap between benign (blue) and malicious (red) samples indicates that the 384-dimensional embedding space provides less discriminative power.

As illustrated in Figures 6–8, the PCA plots show that benign and malicious samples occupy overlapping regions, but OpenAI embeddings exhibit slightly clearer separation and higher explained variance (around 17.0%) compared with GTE-large (16.3%) and MiniLM-L6-v2 (16.8%). The t-SNE visualisations reveal multiple clusters within each class, reflecting different subtypes of benign interactions and IPIAs. Benign and malicious samples form distinct clusters with overlap mainly at the boundaries. UMAP visualisations show the most distinct separation between benign and malicious regions, especially for the OpenAI embeddings. This supports the quantitative results and suggests that the underlying

embedding space learned by OpenAI is particularly well-structured for separating IPIA from benign cases. These visualisations provide interpretability and insights into why certain embedding models outperform others: better-separated clusters in the embedding space directly translate to easier classification and higher detection accuracy.

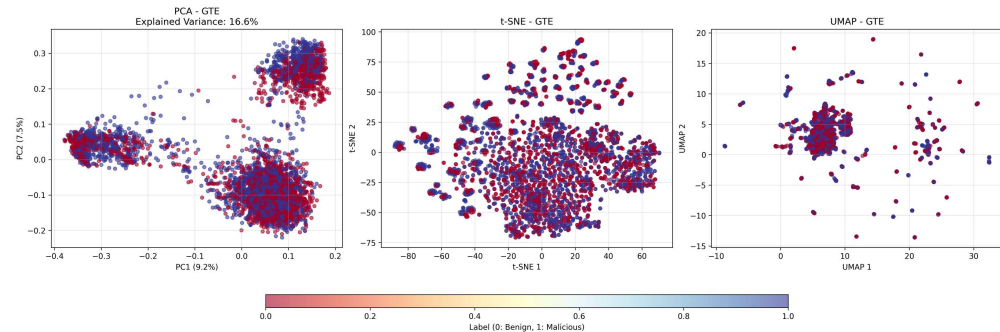


Figure 7. PCA, t-SNE, and UMAP visualisations for GTE-large embeddings. Benign (blue) and malicious (red) samples form overlapping but distinguishable clusters, with UMAP showing clearer separation. This intermediate level of separation aligns with GTE-large’s mid-tier performance.

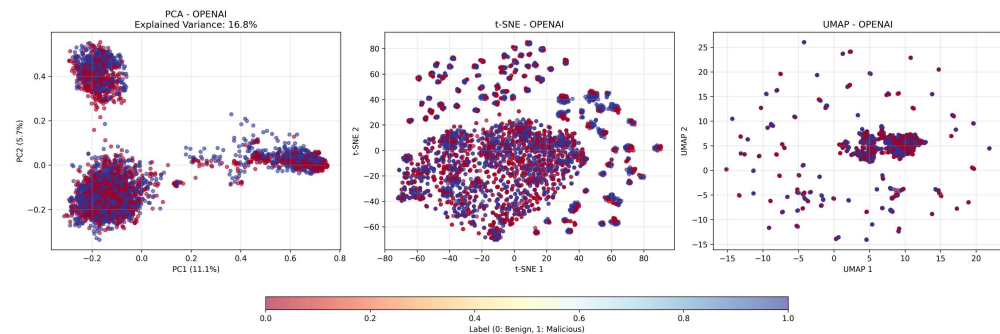


Figure 8. PCA, t-SNE, and UMAP visualisations for OpenAI text-embedding-3-small embeddings. Benign and malicious samples are most clearly separated in this embedding space, particularly under UMAP, which supports strong quantitative performance. The distinct clustering demonstrates that OpenAI embeddings encode semantic differences between benign and malicious contexts most effectively.

The PCA/t-SNE/UMAP projections are included as qualitative visual aids to support interpretation of the embedding space and to complement the quantitative classification results (accuracy, F1, ROC-AUC, and PR-AUC), while cluster-quality measures (e.g., purity) could further quantify separability in the projected space, such metrics are sensitive to projection hyperparameters and do not necessarily reflect separability in the original high-dimensional embedding space. We therefore treat these visualisations as interpretability tools and leave a more systematic cluster-quality analysis as future work.

4.8. Interpretation and Deployment Implications

Overall, the results demonstrate that embedding-based classifiers can serve as an effective and practical detection layer for Indirect Prompt Injection Attacks. The strong performance across multiple model combinations indicates that the approach is robust and does not depend on a single embedding or classifier choice, although OpenAI-XGBoost provide the best performance.

From a deployment perspective, the very low inference times (on the order of microseconds per sample) mean that the detector can be integrated into LLM pipelines as a preprocessing or parallel-checking step without significant latency overhead. The detector can score user-content pairs before the main LLM call, flagging suspicious inputs for blocking or human review. In systems that already employ prevention mechanisms

such as spotlighting, FATH, or system-level controls, the proposed detector can serve as an additional semantic gate, providing an auditable signal indicating whether a given user intent-external content pair resembles known IPIA patterns. The combination of high accuracy, low latency, and model-agnostic design makes this approach particularly suitable for real-world security applications where both effectiveness and operational feasibility are critical.

4.9. Comparison with Existing IPIA Defence and Detection Approaches

Existing defences against indirect prompt injection can be broadly grouped into three categories: (i) prevention mechanisms that aim to constrain how untrusted content influences the model; (ii) rule-based detectors that flag suspicious instructions using heuristic policies; (iii) model-internal approaches that detect injection behaviour using signals from within the LLM. Our method complements these lines of work by providing a lightweight, model-agnostic semantic detector that operates externally in the embedding space.

Prevention mechanisms (e.g., Signed-Prompt and spotlighting). Prevention-focused approaches such as Signed-Prompt and spotlighting aim to reduce instruction ambiguity and prioritise trusted control signals over untrusted content [19,20]. In addition, system-level defences based on information-flow control can restrict the influence of untrusted inputs on downstream actions [21]. While effective, such approaches may require architectural changes, enforcement infrastructure, or deployment-specific integration. In contrast, our approach can be inserted as an external pre-processing gate without modifying the underlying LLM.

Rule-based detection frameworks, such as Palisade, rely on heuristic checks and predefined content rules. They flag suspicious instructions in LLM application inputs based on these manual criteria [23]. These methods can be effective for known injection patterns and interpretable security policies; however, they may require ongoing rule maintenance and can be less robust to paraphrasing or semantically implicit attacks. Our embedding-based classifier captures semantic inconsistency between user intent and external content, enabling detection beyond surface-level string patterns.

Model-internal or attention-based detection methods, such as Attention Tracker, operate within the model to identify malicious inputs. These approaches analyze LLM attention behaviour at inference time to detect Prompt Injection Attacks [24]. While promising, such methods typically require access to model internals and may be less portable across different LLM providers and deployment settings. Our detector remains model-agnostic because it relies only on external embeddings and a lightweight classifier, making it easier to deploy across heterogeneous LLM pipelines.

Overall, our detector provides a scalable semantic filtering layer that complements prevention mechanisms and other detection strategies in a defence-in-depth setting. It can be combined with prevention approaches (e.g., Signed-Prompt/spotlighting) to reduce instruction ambiguity and with rule-based or model-internal detectors to improve coverage against diverse IPIA patterns [19–21,23,24].

5. Conclusions and Future Work

IPIAs pose a serious threat to LLM-integrated systems, especially in settings where models consume untrusted external content and can trigger high-impact actions. In this paper, we propose an embedding-based detection approach that treats IPIA detection as a semantic context analysis problem between user intent and external content. Using the BIPIA benchmark as a source of malicious examples and generating a matching set of benign user-content pairs with a state-of-the-art LLM, we constructed a balanced dataset of 70,000 instances. We systematically evaluated nine configurations that combined three text-

embedding models (OpenAI text-embedding-3-small, GTE-large, and MiniLM-L6-v2) with three tree-based classifiers (Random Forest, XGBoost, and LightGBM). The best-performing configuration, OpenAI embeddings with XGBoost, achieves an accuracy of 97.7% and an F1-score of 0.977, with ROC-AUC and PR-AUC values close to 1.0, while maintaining inference times on the order of microseconds per sample. Compared with existing work, the proposed approach offers three main advantages: it is model-agnostic and operates entirely on embeddings without requiring access to LLM internals; it explicitly models the joint semantic context of user intent and external content rather than relying solely on syntactic rules; finally, it provides a systematic comparison of multiple embedding and classifier combinations, offering practical guidance for practitioners deploying IPIA detectors in LLM-integrated systems.

The proposed semantic-embedding-based detector can be deployed as a pre-processing gate in LLM-integrated systems, such as RAG pipelines, enterprise assistants, or tool-using agents that ingest untrusted external content. By evaluating user–content pairs before they reach the LLM, it can proactively flag potentially malicious or misleading inputs, enabling actions such as blocking, sanitizing, or rerouting for safer handling, all with negligible latency overhead. This approach helps maintain data integrity and mitigate unintended model behaviours in real-world operational contexts. Future extensions could include handling multimodal inputs, implementing adaptive learning to detect emerging prompt-injection patterns, and integrating fine-grained risk scoring or automated content rewriting, further enhancing the robustness, safety, and practical usability of LLM systems in dynamic environments [14,16,17]. Nevertheless, this work has several limitations that open avenues for future research. The dataset relies on BIPIA and synthetically generated benign examples, which may not fully match real-world user–content distributions. In addition, as a single-shot embedding-based detector, the method may be challenged by adaptive attackers who iteratively rewrite injected instructions, particularly in interactive agent settings [17,18]. Future work should validate the approach on external organic datasets and evaluate robustness under basic evasion transformations (e.g., paraphrasing and formatting changes) [18]. Additionally, future work could quantify the structure of the embedding space using cluster-quality metrics, while also performing more extensive ablation studies to assess the contribution of individual components, such as embeddings based solely on user intent or external content. We also plan to report standard deviations across multiple experimental runs for all evaluation metrics providing a more comprehensive assessment of the stability of the results and their statistical robustness. We finally recommend deploying the detector as part of a defence-in-depth strategy alongside complementary prevention and system-level mechanisms [19–21].

Author Contributions: Conceptualisation, M.A., M.T. and S.B.; methodology, M.A.; software, M.A.; validation, M.A., M.T. and S.B.; formal analysis, M.A.; Investigation, M.A.; data curation, M.A., M.T. and S.B.; writing—original draft preparation, M.A.; writing—review and editing, M.T. and S.B.; visualisation, M.A.; supervision, M.T. and S.B.; project administration, M.T. and S.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The complete source code for the proposed embedding-based detection framework, including data preprocessing, embedding extraction, classifier training, and evaluation scripts, is publicly available on GitHub at <https://github.com/Abu-Hussain/Embedding-Based-Detection-of-Indirect-Prompt-Injection-Attacks-in-Large-Language-Models> (accessed on 15 January 2026). The extended BIPIA dataset, comprising both the original malicious samples and the generated benign samples used in this study, is hosted on Hugging Face at <https://huggingface.co/datasets/MAlmasabi/Indirect-Prompt-Injection-BIPIA-GPT> (accessed on 15 January 2026). All resources are

released under an open-source license for academic and research purposes, facilitating reproducibility and enabling further investigation in this domain.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Brown, T.B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language Models are Few-Shot Learners. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 1877–1901. [\[CrossRef\]](#)
2. Chowdhery, A.; Narang, S.; Devlin, J.; Bosma, M.; Mishra, G.; Roberts, A.; Barham, P.; Chung, H.W.; Sutton, C.; Gehrmann, S.; et al. PaLM: Scaling Language Modeling with Pathways. *arXiv* **2022**, arXiv:2204.02311. [\[CrossRef\]](#)
3. OpenAI; Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; et al. GPT-4 Technical Report. *arXiv* **2023**, arXiv:2303.08774. [\[CrossRef\]](#)
4. Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. LLaMA: Open and Efficient Foundation Language Models. *arXiv* **2023**, arXiv:2302.13971. [\[CrossRef\]](#)
5. Alatise, T.I.; Nottidge, O.E. Threat Detection and Response with SIEM System. *Int. J. Comput. Sci. Inf. Secur.* **2024**, *22*, 36–38. [\[CrossRef\]](#)
6. Yao, Y.; Duan, J.; Xu, K.; Cai, Y.; Sun, Z.; Zhang, Y. A survey on large language model (LLM) security and privacy: The Good, The Bad, and The Ugly. *High-Confid. Comput.* **2024**, *4*, 100211. [\[CrossRef\]](#)
7. Brohi, S.; Mastoi, Q.u.a.; Jhanjhi, N.Z.; Pillai, T.R. A Research Landscape of Agentic AI and Large Language Models: Applications, Challenges and Future Directions. *Algorithms* **2025**, *18*, 499. [\[CrossRef\]](#)
8. Li, M.Q.; Fung, B.C. Security Concerns for Large Language Models: A Survey. *J. Inf. Secur. Appl.* **2025**, *95*, 104284. [\[CrossRef\]](#)
9. Kumar, P. Adversarial Attacks and Defenses for Large Language Models (LLMs): Methods, Frameworks & Challenges. *Int. J. Multimed. Inf. Retr.* **2024**, *13*, 26. [\[CrossRef\]](#)
10. Sheng, Z.; Chen, Z.; Gu, S.; Huang, H.; Gu, G.; Huang, J. LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights. *ACM Comput. Surv.* **2025**, *58*, 1–35. [\[CrossRef\]](#)
11. Hamid, R.; Brohi, S. A Review of Large Language Models in Healthcare: Taxonomy, Threats, Vulnerabilities, and Framework. *Big Data Cogn. Comput.* **2024**, *8*, 161. [\[CrossRef\]](#)
12. Perez, F.; Ribeiro, I. Ignore Previous Prompt: Attack Techniques for Language Models. *arXiv* **2022**, arXiv:2211.09527. [\[CrossRef\]](#)
13. Liu, Y.; Jia, Y.; Geng, R.; Jia, J.; Gong, N.Z. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. *arXiv* **2025**, arXiv:2310.12815. [\[CrossRef\]](#)
14. Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; Fritz, M. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*; Association for Computing Machinery: New York, NY, USA; pp. 79–90. [\[CrossRef\]](#)
15. Willison, S. Multi-Modal Prompt Injection Image Attacks Against GPT-4V. 2023. Available online: <https://simonwillison.net/2023/Oct/14/multi-modal-prompt-injection> (accessed on 27 May 2025).
16. Yi, J.; Xie, Y.; Zhu, B.; Kiciman, E.; Sun, G.; Xie, X.; Wu, F. Benchmarking and Defending against Indirect Prompt Injection Attacks on Large Language Models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*; Association for Computing Machinery: New York, NY, USA; pp. 1809–1820. [\[CrossRef\]](#)
17. Zhan, Q.; Liang, Z.; Ying, Z.; Kang, D. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. *arXiv* **2024**, arXiv:2403.02691. [\[CrossRef\]](#)
18. Zhan, Q.; Fang, R.; Panchal, H.S.; Kang, D. Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents. *arXiv* **2025**, arXiv:2503.00061. [\[CrossRef\]](#)
19. Suo, X. Signed-Prompt: A New Approach to Prevent Prompt Injection Attacks Against LLM-Integrated Applications. *arXiv* **2024**, arXiv:2401.07612. [\[CrossRef\]](#)
20. Hines, K.; Lopez, G.; Hall, M.; Zarfati, F.; Zunger, Y.; Kiciman, E. Defending Against Indirect Prompt Injection Attacks With Spotlighting. *arXiv* **2024**, arXiv:2403.14720. [\[CrossRef\]](#)
21. Wu, F.; Cecchetti, E.; Xiao, C. System-Level Defense against Indirect Prompt Injection Attacks: An Information Flow Control Perspective. *arXiv* **2024**, arXiv:2409.19091. [\[CrossRef\]](#)
22. Wang, J.; Wu, F.; Li, W.; Pan, J.; Suh, E.; Mao, Z.M.; Chen, M.; Xiao, C. FATH: Authentication-based Test-time Defense against Indirect Prompt Injection Attacks. *arXiv* **2024**, arXiv:2410.21492. [\[CrossRef\]](#)
23. Kokkula, S.; R, S.; R, N.; Aashishkumar; Divya, G. Palisade—Prompt Injection Detection Framework. *arXiv* **2024**, arXiv:2410.21146. [\[CrossRef\]](#)
24. Hung, K.H.; Ko, C.Y.; Rawat, A.; Chung, I.H.; Hsu, W.H.; Chen, P.Y. Attention Tracker: Detecting Prompt Injection Attacks in LLMs. *arXiv* **2025**, arXiv:2411.00348. [\[CrossRef\]](#)

25. Ayub, M.A.; Majumdar, S. Embedding-based classifiers can detect Prompt Injection Attacks. *arXiv* **2024**, arXiv:2410.22284. [CrossRef]
26. OpenAI. New Embedding Models and API Updates. OpenAI Documentation. 2024. Available online: <https://openai.com/index/new-embedding-models-and-api-updates/> (accessed on 6 June 2025).
27. Li, Z.; Zhang, X.; Zhang, Y.; Long, D.; Xie, P.; Zhang, M. Towards General Text Embeddings With Multi-Stage Contrastive Learning. *arXiv* **2023**, arXiv:2308.03281. [CrossRef]
28. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2019. [CrossRef]
29. Breiman, L. Random Forests. *Mach. Learn.* **2001**, *45*, 5–32. [CrossRef] [PubMed]
30. Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*; Association for Computing Machinery: New York, NY, USA, 2016; pp. 785–794. [CrossRef]
31. Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.Y. LightGBM: A highly efficient gradient boosting decision tree. In *Proceedings of the 31st International Conference on Neural Information Processing Systems; NIPS'17*; Curran Associates Inc.: Red Hook, NY, USA, 2017; pp. 3149–3157. Available online: <https://dl.acm.org/doi/10.5555/3294996.3295074> (accessed on 15 June 2025).
32. Maaten, L.v.d.; Hinton, G. Visualizing Data Using t-SNE. *J. Mach. Learn. Res.* **2008**, *9*, 2579–2605. Available online: <http://jmlr.org/papers/v9/vandermaten08a.html> (accessed on 15 June 2025)
33. McInnes, L.; Healy, J.; Saul, N.; Großberger, L. UMAP: Uniform Manifold Approximation and Projection. *J. Open Source Softw.* **2018**, *3*, 861. [CrossRef]
34. Mathew, E.S. Enhancing Security in Large Language Models: A Comprehensive Review of Prompt Injection Attacks and Defenses. *J. Artif. Intell.* **2025**, *7*, 347–363. [CrossRef]
35. Zou, A.; Wang, Z.; Carlini, N.; Nasr, M.; Kolter, J.Z.; Fredrikson, M. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv* **2023**, arXiv:2307.15043. [CrossRef]
36. Shi, J.; Yuan, Z.; Liu, Y.; Huang, Y.; Zhou, P.; Sun, L.; Gong, N.Z. Optimization-Based Prompt Injection Attack to LLM-as-a-Judge. *arXiv* **2024**, arXiv:2403.17710. [CrossRef]
37. Huang, Y.; Wang, C.; Jia, X.; Guo, Q.; Juefei-Xu, F.; Zhang, J.; Liu, Y.; Pu, G. Efficient Universal Goal Hijacking with Semantics-guided Prompt Organization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vienna, Austria; Association for Computational Linguistics: Stroudsburg, PA, USA, 2025; pp. 5796–5816. [CrossRef]
38. Heibel, J.; Lowd, D. MaPPing Your Model: Assessing the Impact of Adversarial Attacks on LLM-based Programming Assistants. *arXiv* **2024**, arXiv:2407.11072. [CrossRef]
39. Xue, J.; Zheng, M.; Hu, Y.; Liu, F.; Chen, X.; Lou, Q. BadRAG: Identifying Vulnerabilities in Retrieval Augmented Generation of Large Language Models. *arXiv* **2024**, arXiv:2406.00083. [CrossRef]
40. Liang, X.; Niu, S.; Li, Z.; Zhang, S.; Wang, H.; Xiong, F.; Fan, Z.; Tang, B.; Zhao, J.; Yang, J.; et al. SafeRAG: Benchmarking Security in Retrieval-Augmented Generation of Large Language Model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*; Che, W., Nabende, J., Shutova, E., Pilehvar, M.T., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2025. [CrossRef]
41. Zhao, W.; Gupta, A.; Chung, T.; Huang, J. SPC: Soft Prompt Construction for Cross Domain Generalization. In *Proceedings of the 8th Workshop on Representation Learning for NLP (RepL4NLP 2023)*; Can, B., Mozes, M., Cahyawijaya, S., Saphra, N., Kassner, N., Ravfogel, S., Ravichander, A., Zhao, C., Augenstein, I., Rogers, A., et al., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2023; pp. 118–130. [CrossRef]
42. Bommasani, R.; Hudson, D.A.; Adeli, E.; Altman, R.; Arora, S.; von Arx, S.; Bernstein, M.S.; Bohg, J.; Bosselut, A.; Brunskill, E.; et al. On the Opportunities and Risks of Foundation Models. *arXiv* **2022**, arXiv:2108.07258. [CrossRef]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.